

Linux Production Shell Scripts

PART-2

Data Cleanup Scripts

Here are more **data cleanup shell scripts** to cover a variety of cleanup scenarios:

1. Cleanup Temporary Files

This script cleans up temporary files from the /tmp directory.

```
bash
Copy code
#!/bin/bash

# Clean up temporary files
log_file="/var/log/tmp_cleanup.log"
log_message() {
    echo "$(date +%Y-%m-%d %H:%M:%S) - $1" | tee -a "$log_file"
}

log_message "Starting cleanup of /tmp directory."

find /tmp -type f -atime +7 -exec rm -f {} \; && log_message "Old
temporary files deleted."
find /tmp -type d -empty -delete && log_message "Empty directories
removed."

log_message "Temporary file cleanup completed."
```

2. Clear Old Log Files

Deletes old log files from /var/log that are no longer needed.

```
bash
Copy code
#!/bin/bash
```

```
# Delete old log files
LOG_DIR="/var/log"
DAYS_OLD=30
```

```
echo "Cleaning log files older than $DAYS_OLD days in
$LOG_DIR."
find "$LOG_DIR" -type f -name "*.log" -mtime +$DAYS_OLD -exec
rm -f {} \;
echo "Log file cleanup completed."
```

3. Cleanup User-Specific Cache Files

Removes .cache files from user home directories.

```
bash
Copy code
#!/bin/bash
```

```
# Cleanup user cache
for dir in /home/*; do
    if [ -d "$dir/.cache" ]; then
        echo "Cleaning cache in $dir"
        rm -rf "$dir/.cache/*"
    fi
done
echo "User cache cleanup completed."
```

4. Remove Large Files (Above a Certain Size)

Identifies and removes files larger than 500MB in a target directory.

```
bash
Copy code
```

```
#!/bin/bash
```

```
TARGET_DIR="/path/to/dir"  
MAX_SIZE=500M
```

```
echo "Removing files larger than $MAX_SIZE in $TARGET_DIR."  
find "$TARGET_DIR" -type f -size +$MAX_SIZE -exec rm -f {} \;  
echo "Large file cleanup completed."
```

5. Automated Old Backup Cleanup

Cleans up backup files older than 60 days.

```
bash  
Copy code  
#!/bin/bash
```

```
# Cleanup old backups  
BACKUP_DIR="/backups"  
DAYS_OLD=60
```

```
echo "Cleaning up backups older than $DAYS_OLD days in  
$BACKUP_DIR."  
find "$BACKUP_DIR" -type f -mtime +$DAYS_OLD -exec rm -f {} \;  
echo "Backup cleanup completed."
```

6. Disk Space Monitoring with Cleanup

Automatically cleans files if disk usage exceeds 90%.

```
bash  
Copy code  
#!/bin/bash
```

```
THRESHOLD=90  
DIR_TO_CLEAN="/var/tmp"
```

```
usage=$(df "$DIR_TO_CLEAN" | awk 'NR==2 {print $5}' | sed 's/%//')
```

```
if [ "$usage" -gt "$THRESHOLD" ]; then
    echo "Disk usage is above $THRESHOLD%. Cleaning up $DIR_TO_CLEAN."
    find "$DIR_TO_CLEAN" -type f -atime +7 -exec rm -f {} \;
    echo "Cleanup completed."
else
    echo "Disk usage is below threshold. No cleanup required."
fi
```

7. Cleanup Orphaned Docker Images

Removes unused Docker images to free up space.

```
bash
Copy code
#!/bin/bash

echo "Cleaning up unused Docker images."
docker image prune -af
echo "Docker image cleanup completed."
```

8. Database Log Cleanup

Deletes old database log files. Customize the path to your database logs.

```
bash
Copy code
#!/bin/bash

DB_LOG_DIR="/var/lib/mysql/logs"
DAYS_OLD=15
```

```
echo "Cleaning database logs older than $DAYS_OLD days in  
$DB_LOG_DIR."  
find "$DB_LOG_DIR" -type f -mtime +$DAYS_OLD -exec rm -f {} \;  
echo "Database log cleanup completed."
```

Website Uptime Checker Scripts

1. Basic Website Uptime Checker

This script checks if the website is up by sending an HTTP request and logging the response.

```
bash  
Copy code  
#!/bin/bash  
  
# Basic Website Uptime Checker  
# Description: This script checks if the website is up or down based  
on HTTP status code.  
  
WEBSITE_URL="https://example.com"  
LOG_FILE="/var/log/website_uptime.log"  
  
# Log message function  
log_message() {  
    echo "$(date +%Y-%m-%d %H:%M:%S) - $1" | tee -a  
"$LOG_FILE"  
}  
  
# Website uptime check  
HTTP_STATUS=$(curl -o /dev/null -s -w "%{http_code}"  
"$WEBSITE_URL")  
if [ "$HTTP_STATUS" -eq 200 ]; then  
    log_message "Website $WEBSITE_URL is UP. Status:  
$HTTP_STATUS."  
else
```

```
    log_message "ALERT: Website $WEBSITE_URL is DOWN.  
Status: $HTTP_STATUS."  
fi
```

2. Website Uptime Checker with Custom Interval

This script checks the website at regular intervals (set by you) and logs the status.

```
bash
```

Copy code

```
#!/bin/bash
```

```
# Website Uptime Checker with Custom Interval
```

```
# Description: This script checks the uptime of a website at  
specified intervals.
```

```
WEBSITE_URL="https://example.com"
```

```
LOG_FILE="/var/log/website_uptime.log"
```

```
CHECK_INTERVAL=60 # Interval in seconds
```

```
log_message() {  
    echo "$(date +%Y-%m-%d %H:%M:%S) - $1" | tee -a  
"$LOG_FILE"  
}
```

```
while true; do
```

```
    HTTP_STATUS=$(curl -o /dev/null -s -w "%{http_code}"  
"$WEBSITE_URL")
```

```
    if [ "$HTTP_STATUS" -eq 200 ]; then
```

```
        log_message "Website $WEBSITE_URL is UP. Status:  
$HTTP_STATUS."
```

```
    else
```

```
        log_message "ALERT: Website $WEBSITE_URL is DOWN.  
Status: $HTTP_STATUS."
```

```
    fi
```

```
    sleep "$CHECK_INTERVAL"
```

```
done
```

3. Website Uptime Checker with Email Alerts

This script sends an email notification if the website is down. You need to configure an email server (e.g., sendmail or mail command).

```
bash
```

```
Copy code
```

```
#!/bin/bash
```

```
# Website Uptime Checker with Email Alerts
```

```
# Description: This script checks the uptime of a website and sends an email if it's down.
```

```
WEBSITE_URL="https://example.com"
```

```
LOG_FILE="/var/log/website_uptime.log"
```

```
EMAIL="your-email@example.com"
```

```
log_message() {
```

```
    echo "$(date +%Y-%m-%d %H:%M:%S) - $1" | tee -a  
"$LOG_FILE"
```

```
}
```

```
HTTP_STATUS=$(curl -o /dev/null -s -w "%{http_code}"  
"$WEBSITE_URL")
```

```
if [ "$HTTP_STATUS" -eq 200 ]; then
```

```
    log_message "Website $WEBSITE_URL is UP. Status:  
$HTTP_STATUS."
```

```
else
```

```
    log_message "ALERT: Website $WEBSITE_URL is DOWN.  
Status: $HTTP_STATUS."
```

```
    echo "ALERT: Website $WEBSITE_URL is DOWN. Status:  
$HTTP_STATUS." | mail -s "Website Down Alert" "$EMAIL"
```

```
fi
```

4. Multi-Site Uptime Checker

Checks the uptime of multiple websites and logs the status for each.

bash

Copy code

#!/bin/bash

Multi-Site Uptime Checker

Description: This script checks the uptime of multiple websites and logs the status for each.

```
SITES=("https://example1.com" "https://example2.com"
"https://example3.com")
```

```
LOG_FILE="/var/log/multi_site_uptime.log"
```

```
log_message() {
    echo "$(date +%Y-%m-%d %H:%M:%S) - $1" | tee -a
"$LOG_FILE"
}
```

```
for SITE in "${SITES[@]"; do
    HTTP_STATUS=$(curl -o /dev/null -s -w "%{http_code}" "$SITE")
    if [ "$HTTP_STATUS" -eq 200 ]; then
        log_message "$SITE is UP. Status: $HTTP_STATUS."
    else
        log_message "ALERT: $SITE is DOWN. Status:
$HTTP_STATUS."
    fi
done
```

5. Website Uptime Checker with Response Time Logging

This script also logs the response time of the website.

bash

Copy code

```
#!/bin/bash
```

```
# Website Uptime Checker with Response Time Logging  
# Description: This script checks if the website is up and logs the  
response time.
```

```
WEBSITE_URL="https://example.com"  
LOG_FILE="/var/log/website_uptime.log"
```

```
log_message() {  
    echo "$(date +%Y-%m-%d %H:%M:%S) - $1" | tee -a  
"$LOG_FILE"  
}
```

```
START_TIME=$(date +%s.%N)  
HTTP_STATUS=$(curl -o /dev/null -s -w "%{http_code}"  
"$WEBSITE_URL")  
END_TIME=$(date +%s.%N)
```

```
RESPONSE_TIME=$(echo "$END_TIME - $START_TIME" | bc)
```

```
if [ "$HTTP_STATUS" -eq 200 ]; then  
    log_message "$WEBSITE_URL is UP. Status: $HTTP_STATUS.  
Response Time: ${RESPONSE_TIME}s."  
else  
    log_message "ALERT: $WEBSITE_URL is DOWN. Status:  
$HTTP_STATUS."  
fi
```

6. Website Uptime Checker with Dashboard Integration

This script can integrate with a dashboard (e.g., Grafana or Prometheus) by sending status data in JSON format for further visualization.

```
bash
```

Copy code

```
#!/bin/bash
```

```
# Website Uptime Checker with Dashboard Integration
```

```
# Description: This script checks the website and sends uptime data  
in JSON format for dashboard monitoring.
```

```
WEBSITE_URL="https://example.com"
```

```
DASHBOARD_API="http://your-dashboard.com/api/uptime"
```

```
LOG_FILE="/var/log/website_uptime.log"
```

```
log_message() {  
    echo "$(date +%Y-%m-%d %H:%M:%S) - $1" | tee -a  
"$LOG_FILE"  
}
```

```
HTTP_STATUS=$(curl -o /dev/null -s -w "%{http_code}"
```

```
"$WEBSITE_URL")
```

```
CURRENT_TIME=$(date +%Y-%m-%d %H:%M:%S)
```

```
if [ "$HTTP_STATUS" -eq 200 ]; then
```

```
    JSON_DATA="{\"website\": \"$WEBSITE_URL\", \"status\":  
\"UP\", \"time\": \"$CURRENT_TIME\"}"
```

```
    curl -X POST -H "Content-Type: application/json" -d  
"$JSON_DATA" "$DASHBOARD_API"
```

```
    log_message "$WEBSITE_URL is UP. Status: $HTTP_STATUS."  
else
```

```
    JSON_DATA="{\"website\": \"$WEBSITE_URL\", \"status\":  
\"DOWN\", \"time\": \"$CURRENT_TIME\"}"
```

```
    curl -X POST -H "Content-Type: application/json" -d  
"$JSON_DATA" "$DASHBOARD_API"
```

```
    log_message "ALERT: $WEBSITE_URL is DOWN. Status:  
$HTTP_STATUS."
```

```
fi
```

System Information Shell Scripts

1. Basic System Information

This script displays general system information, including OS, kernel version, and architecture.

```
bash
```

```
Copy code
```

```
#!/bin/bash
```

```
# Basic System Information
```

```
# Description: This script displays basic system information like OS, kernel version, and architecture.
```

```
echo "Operating System: $(uname -o)"
```

```
echo "Kernel Version: $(uname -r)"
```

```
echo "Machine Architecture: $(uname -m)"
```

```
echo "Host Name: $(hostname)"
```

```
echo "System Uptime: $(uptime -p)"
```

```
echo "Total Users Logged In: $(who -q | head -n 1)"
```

2. CPU Usage and Info

This script shows detailed information about the CPU, including cores, model, and current usage.

```
bash
```

```
Copy code
```

```
#!/bin/bash
```

```
# CPU Usage and Info
```

```
# Description: This script shows CPU information, including cores and usage statistics.
```

```
echo "CPU Information:"
```

```
lscpu
```

```
echo ""
```

```
echo "Current CPU Usage:"  
top -bn1 | grep "Cpu(s)"
```

3. Memory Usage and Info

Displays the total, used, and free memory, along with swap usage.

```
bash  
Copy code  
#!/bin/bash
```

```
# Memory Usage and Info  
# Description: This script shows the current memory usage along  
with swap statistics.
```

```
echo "Memory Information:"  
free -h  
echo ""
```

```
echo "Swap Information:"  
swapon --show
```

4. Disk Usage and File System Information

This script shows the disk usage by partition and the file system details.

```
bash  
Copy code  
#!/bin/bash
```

```
# Disk Usage and File System Information  
# Description: This script shows the disk usage and file system  
information.
```

```
echo "Disk Usage:"
```

```
df -h
echo ""
```

```
echo "File System Details:"
lsblk
```

5. System Load and Process Information

Shows system load averages and active processes.

```
bash
Copy code
#!/bin/bash
```

```
# System Load and Process Information
# Description: This script displays system load averages and the top
processes.
```

```
echo "System Load Averages (1, 5, 15 minutes):"
uptime | awk -F'load average:' '{ print $2 }'
echo ""
```

```
echo "Top 5 Processes by CPU Usage:"
ps aux --sort=-%cpu | head -n 6
```

6. Network Information

Displays network interfaces, IP addresses, and connection stats.

```
bash
Copy code
#!/bin/bash
```

```
# Network Information
# Description: This script displays network interface and connection
information.
```

```
echo "Network Interfaces and IP Addresses:"  
ip a  
echo ""
```

```
echo "Active Network Connections:"  
ss -tuln
```

7. System Hardware Information (Using lshw)

Shows detailed hardware information, such as CPU, RAM, and storage.

```
bash  
Copy code  
#!/bin/bash
```

```
# System Hardware Information  
# Description: This script displays detailed hardware info using lshw.
```

```
echo "Hardware Information:"  
sudo lshw -short
```

8. Disk I/O Performance (Using iostat)

Displays real-time I/O performance statistics.

```
bash  
Copy code  
#!/bin/bash
```

```
# Disk I/O Performance  
# Description: This script shows disk input/output statistics using  
iostat.
```

```
echo "Disk I/O Performance:"  
iostat -dx 1 10
```

9. Top 5 Largest Files in System

This script lists the top 5 largest files across the system.

```
bash
Copy code
#!/bin/bash

# Top 5 Largest Files
# Description: This script finds the top 5 largest files in the system.

echo "Top 5 Largest Files in System:"
find / -type f -exec du -h {} + | sort -rh | head -n 5
```

10. System Boot Time

This script shows the system's boot time.

```
bash
Copy code
#!/bin/bash

# System Boot Time
# Description: This script displays the system's boot time.

echo "System Boot Time:"
uptime -s
```

11. System Resource Usage Report (Custom Report)

A combined report showing CPU, memory, and disk usage.

```
bash
Copy code
#!/bin/bash

# System Resource Usage Report
```

Description: This script generates a combined report of CPU, memory, and disk usage.

```
echo "----- SYSTEM RESOURCE USAGE REPORT -----"
```

```
echo "CPU Information:"
```

```
lscpu
```

```
echo ""
```

```
echo "Memory Usage:"
```

```
free -h
```

```
echo ""
```

```
echo "Disk Usage:"
```

```
df -h
```

```
echo ""
```

```
echo "Active Processes (Top 5):"
```

```
ps aux --sort=-%cpu | head -n 6
```

CPU Usage Tracker Scripts

Here are several **CPU Usage Tracker Scripts** that will help you monitor the CPU usage over time and alert you when the CPU usage exceeds a specified threshold:

1. Basic CPU Usage Tracker

This script checks the CPU usage and logs it to a file with the current timestamp.

```
bash
```

```
Copy code
```

```
#!/bin/bash
```

```
# Basic CPU Usage Tracker
```

Description: This script checks the current CPU usage and logs it to a file with the timestamp.

```
LOG_FILE="/var/log/cpu_usage.log"
```

```
log_cpu_usage() {  
    CPU_USAGE=$(top -bn1 | grep "Cpu(s)" | sed "s/.*, *\([0-9.]*\)%"*  
id.*\1/" | awk '{print 100 - $1}')
```

```
    TIMESTAMP=$(date "+%Y-%m-%d %H:%M:%S")  
    echo "$TIMESTAMP - CPU Usage: $CPU_USAGE%" >>  
"$LOG_FILE"
```

```
}
```

```
# Run every minute  
while true; do  
    log_cpu_usage  
    sleep 60  
done
```

2. CPU Usage Alert Script

This script checks the CPU usage and sends an email alert if the usage exceeds a certain threshold.

```
bash  
Copy code  
#!/bin/bash
```

```
# CPU Usage Alert Script
```

```
# Description: This script checks the CPU usage and sends an  
email if the usage exceeds 90%.
```

```
THRESHOLD=90
```

```
EMAIL="your-email@example.com"
```

```
LOG_FILE="/var/log/cpu_usage_alerts.log"
```

```
log_alert() {  
    TIMESTAMP=$(date "+%Y-%m-%d %H:%M:%S")
```

```

    echo "$TIMESTAMP - CPU Usage Alert: $1%" >> "$LOG_FILE"
    echo "ALERT: CPU Usage has exceeded $THRESHOLD%.
Current usage: $1%" | mail -s "CPU Usage Alert" "$EMAIL"
}

```

```

CPU_USAGE=$(top -bn1 | grep "Cpu(s)" | sed "s/.*, *\([0-9.]*\)%"*
id.*\1/" | awk '{print 100 - $1}')

```

```

if (( $(echo "$CPU_USAGE > $THRESHOLD" | bc -l) )); then
    log_alert "$CPU_USAGE"
fi

```

3. CPU Usage Monitor with Time Interval

This script continuously monitors CPU usage at a specified time interval and logs the data.

bash

Copy code

#!/bin/bash

CPU Usage Monitor with Time Interval

Description: This script continuously monitors CPU usage at a specified interval (e.g., every 5 seconds).

LOG_FILE="/var/log/cpu_usage_monitor.log"

INTERVAL=5 # In seconds

```

log_cpu_usage() {
    CPU_USAGE=$(top -bn1 | grep "Cpu(s)" | sed "s/.*, *\([0-9.]*\)%"*
id.*\1/" | awk '{print 100 - $1}')
    TIMESTAMP=$(date "+%Y-%m-%d %H:%M:%S")
    echo "$TIMESTAMP - CPU Usage: $CPU_USAGE%" >>
"$LOG_FILE"
}

```

while true; do

log_cpu_usage

```
    sleep "$INTERVAL"
done
```

4. CPU Usage Tracker with Top Processes

This script tracks the CPU usage and lists the top 5 CPU-consuming processes.

```
bash
Copy code
#!/bin/bash
```

```
# CPU Usage Tracker with Top Processes
# Description: This script tracks the overall CPU usage and shows
the top 5 processes consuming the most CPU.
```

```
LOG_FILE="/var/log/cpu_usage_top_processes.log"
```

```
log_cpu_usage() {
    CPU_USAGE=$(top -bn1 | grep "Cpu(s)" | sed "s/.*, *\([0-9.]*\)%"*
id.*\1/" | awk '{print 100 - $1}')
    TIMESTAMP=$(date "+%Y-%m-%d %H:%M:%S")
    echo "$TIMESTAMP - CPU Usage: $CPU_USAGE%" >>
"$LOG_FILE"
    echo "$TIMESTAMP - Top 5 CPU-consuming processes:" >>
"$LOG_FILE"
    ps aux --sort=-%cpu | head -n 6 >> "$LOG_FILE"
}
```

```
# Run every minute
while true; do
    log_cpu_usage
    sleep 60
done
```

5. CPU Usage and Load Average Tracker

This script checks CPU usage and system load averages and logs them with the timestamp.

bash

Copy code

#!/bin/bash

CPU Usage and Load Average Tracker

Description: This script tracks CPU usage and load averages and logs them with timestamps.

LOG_FILE="/var/log/cpu_load_usage.log"

```
log_cpu_load() {  
    CPU_USAGE=$(top -bn1 | grep "Cpu(s)" | sed "s/.*, *\([0-9.]*\)%"*  
id.*\1/" | awk '{print 100 - $1}')  
    LOAD_AVG=$(uptime | awk -F'load average:' '{ print $2 }')  
    TIMESTAMP=$(date "+%Y-%m-%d %H:%M:%S")  
    echo "$TIMESTAMP - CPU Usage: $CPU_USAGE%, Load  
Average: $LOAD_AVG" >> "$LOG_FILE"  
}
```

Run every 5 minutes

while true; do

log_cpu_load

sleep 300

done

6. Continuous CPU Usage Monitoring and Logging

This script logs CPU usage continuously in real-time and generates a report after every hour.

bash

Copy code

#!/bin/bash

Continuous CPU Usage Monitoring and Logging
Description: This script continuously monitors and logs CPU usage every second and generates a report every hour.

```
LOG_FILE="/var/log/continuous_cpu_usage.log"
REPORT_FILE="/var/log/cpu_usage_report_$(date
+'%Y%m%d').log"
```

```
log_cpu_usage() {
    CPU_USAGE=$(top -bn1 | grep "Cpu(s)" | sed "s/.*, *\[0-9.\]*%*
id.*\1/" | awk '{print 100 - $1}')
    TIMESTAMP=$(date "+%Y-%m-%d %H:%M:%S")
    echo "$TIMESTAMP - CPU Usage: $CPU_USAGE%" >>
"$LOG_FILE"
}
```

```
generate_report() {
    echo "----- CPU Usage Report -----" > "$REPORT_FILE"
    awk '{print $1, $2, $3, $4, $5, $6}' "$LOG_FILE" >>
"$REPORT_FILE"
}
```

```
# Run every 1 second and generate report every hour
while true; do
    log_cpu_usage
    sleep 1
    if (( $(date +%M) == 00 )); then
        generate_report
    fi
done
```

7. CPU Usage Tracker with Threshold Notification

This script sends a desktop notification if the CPU usage exceeds a specified threshold.

bash
Copy code

```
#!/bin/bash
```

```
# CPU Usage Tracker with Threshold Notification
```

```
# Description: This script sends a desktop notification if the CPU  
usage exceeds a defined threshold.
```

```
THRESHOLD=80
```

```
NOTIFY_CMD="notify-send"
```

```
LOG_FILE="/var/log/cpu_usage_threshold.log"
```

```
log_and_notify() {
```

```
    CPU_USAGE=$(top -bn1 | grep "Cpu(s)" | sed "s/.*, *\([0-9.]*\)%"*  
id.*\1/" | awk '{print 100 - $1}')
```

```
    TIMESTAMP=$(date "+%Y-%m-%d %H:%M:%S")
```

```
    echo "$TIMESTAMP - CPU Usage: $CPU_USAGE%" >>  
"$LOG_FILE"
```

```
    if (( $(echo "$CPU_USAGE > $THRESHOLD" | bc -l) )); then  
        $NOTIFY_CMD "CPU Usage Alert" "CPU usage has exceeded  
the threshold: $CPU_USAGE%"  
    fi  
}
```

```
# Run every minute
```

```
while true; do
```

```
    log_and_notify
```

```
    sleep 60
```

```
done
```

**FOLLOW PRINCE JOSHI FOR FURTHER CONTENT
LIKE THIS IN FUTURE**